



## Complete Autonomous Agent Development Log & Conversation Transcript

**Developer / Author:** Jeevanthi Kashyap

**Repository:** <https://github.com/Jeevanthikashyap/buildwithgemini-yatra-ai>

**Project Architecture Summary:** Google Agent Development Kit (ADK), Vertex AI Agent Platform (Agent Runtime / Reasoning Engine), Google Cloud Firestore, Cloud Storage, Vertex AI Memory Bank (cross-session memory), Google Omni (gemini-omni-flash-preview video generation), Imagen (gemini-3.1-flash-lite-image postcards), Google Maps Platform Places & Geocoding APIs, Agent Engine Python Sandbox, and A2A Protocol with A2UI v0.8.

### TURN 1 — USER

Download and setup <https://github.com/cszhu/build-with-gemini>

<USER\_SETTINGS\_CHANGE>

The user changed setting **Model Selection** from None to Gemini 3.8 Flash (Low). No need to comment on this change if the user doesn't ask about it. If reporting what model you are, please use a human readable name instead of the exact string.

</USER\_SETTINGS\_CHANGE>

### TURN 2 — ASSISTANT (ANTIGRAVITY)

The **[build-with-gemini]**(<https://github.com/cszhu/build-with-gemini>) repository has been downloaded and configured in your workspace: [ [/config/Desktop/BuildWithGemini](#) ](file:///config/Desktop/BuildWithGemini).

#### Setup & Verification Summary

##### 1. Repository:

- Cloned branch `main` into [ [/config/Desktop/BuildWithGemini](#) ](file:///config/Desktop/BuildWithGemini).
- Workshop skills installed under [ [.agents/skills](#) ](file:///config/Desktop/BuildWithGemini/.agents/skills) ( [pick-your-agent-project](#) , [setup-memory-bank](#) , [enable-a2ui](#) , [build-agent-frontend](#) , [publish-to-github](#) , [record-demo](#) , [build-rag](#) , [troubleshoot-lab-setup](#) ).
- MCP tools configured in [ [mcp\\_config.json](#) ](file:///config/Desktop/BuildWithGemini/.agents/mcp\_config.json) (Firebase + Google Developer Knowledge).

##### 2. Environment & Auth Checks:

- GCP Project:** [qwiklabs-gcp-04-8017249fc17a](#) ✓
- GCP Account:** [antigravity-sa@qwiklabs-gcp-04-8017249fc17a.iam.gserviceaccount.com](#) ✓
- Vertex AI / Agent Platform API:** [aiplatform.googleapis.com](#) is enabled ✓
- CLI Tools:** [agents-cli](#) , [adk](#) , [uv](#) , and [python3](#) are installed and ready ✓
- agents-cli Skills:** Setup executed; registered [google-agents-cli-\\*](#) lifecycle skills ✓
- URL Interceptors:** Fixed and verified [/config/automata/bin/xdg-open](#) & [shim](#) executable shims ✓

Your environment is ready to start building.

**TURN 3 — USER**

Use agents-cli to build a simple agent I can test and run it locally.

Use agents-cli to build a simple agent I can test and run it locally.

Use agents-cli to build a simple agent I can test and run it locally.

**TURN 4 — ASSISTANT (ANTIGRAVITY)**

The following is a <SYSTEM\_MESSAGE> not actually sent by the user. It is provided by the system as important information to pay attention to.

<SYSTEM\_MESSAGE>

[Message] timestamp=2026-09-29T09:47:22Z sender=be4c90de-9c4e-4c03-a3d8-d03d0e174fd7/task-56  
priority=MESSAGE\_PRIORITY\_HIGH content=Task id "be4c90de-9c4e-4c03-a3d8-d03d0e174fd7/task-56" finished with result:

The command exited with code 1.

Output:

<truncated 56 lines>

> assert result is not None

E assert None is not None

tests/integration/test\_server\_e2e.py:76: AssertionError

----- Captured stderr call -----

[2026-09-29 09:47:19 +0000] [1719] [INFO] Started server process [1719]

[2026-09-29 09:47:19 +0000] [1719] [INFO] Waiting for application startup.

[2026-09-29 09:47:19 +0000] [1719] [INFO] Application startup complete.

[2026-09-29 09:47:19 +0000] [1719] [INFO] Uvicorn running on http://127.0.0.1:44605 (Press CTRL+C to quit)

[2026-09-29 09:47:20 +0000] [1719] [INFO] 127.0.0.1:54992 - "GET /livez HTTP/1.1" 200 OK

[2026-09-29 09:47:20 +0000] [1719] [INFO] 127.0.0.1:54992 - "POST /stream\_query HTTP/1.1" 200 OK

----- Captured log call -----

INFO uvicorn.error:server.py:86 Started server process [1719]

INFO uvicorn.error:on.py:53 Waiting for application startup.

INFO uvicorn.error:on.py:71 Application startup complete.

INFO uvicorn.error:server.py:222 Uvicorn running on http://127.0.0.1:44605 (Press CTRL+C to quit)

INFO uvicorn.access:trace.py:61 127.0.0.1:54992 - "GET /livez HTTP/1.1" 200 OK

INFO uvicorn.access:trace.py:61 127.0.0.1:54992 - "POST /stream\_query HTTP/1.1" 200 OK

\_\_\_\_\_ test\_e2e\_query\_without\_streaming \_\_\_\_\_

self = <tests.integration.test\_server\_e2e.E2ETestCase testMethod=test\_e2e\_query\_without\_streaming>

def test\_e2e\_query\_without\_streaming(self):

"""Test sending a query without streaming."""

result = self.client.query(

user\_id="test\_user",

session\_id=self.session\_id,

message="What is the weather in Seattle?",

)

> assert result is not None

E assert None is not None

tests/integration/test\_server\_e2e.py:89: AssertionError

----- Captured stderr call -----

[2026-09-29 09:47:21 +0000] [1719] [INFO] 127.0.0.1:55006 - "POST /query HTTP/1.1" 2

The following is a <SYSTEM\_MESSAGE> not actually sent by the user. It is provided by the system as important information to pay attention to.

<SYSTEM\_MESSAGE>

[Message] timestamp=2026-09-29T09:47:56Z sender=be4c90de-9c4e-4c03-a3d8-d03d0e174fd7/task-69

priority=MESSAGE\_PRIORITY\_HIGH content=Task id "be4c90de-9c4e-4c03-a3d8-d03d0e174fd7/task-69" finished with result:

The command exited with code 0.

Output:

<truncated 30 lines>

tests/integration/test\_server\_e2e.py::E2ETestCase::test\_e2e\_query\_without\_streaming PASSED [ 50%]

tests/integration/test\_server\_e2e.py::E2ETestCase::test\_e2e\_stream\_query PASSED [ 66%]

tests/integration/test\_server\_e2e.py::E2ETestCase::test\_health\_check PASSED [ 83%]

tests/integration/test\_server\_e2e.py::E2ETestCase::test\_read\_root PASSED [100%]

===== warnings summary =====

tests/integration/test\_server\_e2e.py::E2ETestCase::test\_e2e\_query\_without\_streaming

/config/Desktop/BuildWithGemini/demo-agent/.venv/lib/python3.13/site-packages/starlette/formparsers.py:16:

DeprecationWarning: The `python-multipart` library is deprecated and will be removed in a future release. Please use `multipart` instead.

import multipart

tests/integration/test\_server\_e2e.py::E2ETestCase::test\_e2e\_query\_without\_streaming

/config/.local/share/uv/python/cpython-3.13.15-linux-x86\_64-gnu/lib/python3.13/site-

packages/opentelemetry/instrumentation/dependencies.py:4: DeprecationWarning: pkg\_resources is deprecated as an API. See comparison between distribute and setuptools at [https://setuptools.pypa.io/en/latest/pkg\\_resources.html](https://setuptools.pypa.io/en/latest/pkg_resources.html). The pkg\_resources API is scheduled for removal in a future release.

from pkg\_resources import (

tests/integration/test\_server\_e2e.py::E2ETestCase::test\_e2e\_query\_without\_streaming

/config/Desktop/BuildWithGemini/demo-agent/.venv/lib/python3.13/site-packages/pkg\_resources/\_\_init\_\_.py:3147:

DeprecationWarning: Deprecated call to `pkg_resources.declare_namespace('google')`.

Implementing implicit namespace packages (as specified in PEP 420) is preferred to `pkg_resources.declare_namespace`. See <https://setuptools.pypa.io/en/latest/references/keywords.html#namespace-packages>

declare\_namespace(pkg)

tests/integration/test\_server\_e2e.py::E2ETestCase::test\_e2e\_query\_without\_streaming

/config/Desktop/BuildWithGemini/demo-agent/.venv/lib/python3.13/site-packages/pkg\_resources/\_\_init\_\_.py:3147:

DeprecationWarning: Deprecated call to `pkg_resources.declare_namespace('google.cloud')`.

Implementing implicit namespace packages (as specified in PEP 420) is preferred to `pkg_resources.declare_namespace`. See <https://setuptools.pypa.io/en/latest/references/keywords.html#namespace-packages>

declare\_namespace(pkg)

-- Docs: <https://docs.pytest.org/en/stable/how-to/capture-warnings.html>

===== 6 passed, 4 warnings in 20.30s =====

</SYSTEM\_MESSAGE>

A simple agent has been scaffolded, configured, and tested locally using `agents-cli` and the Agent Development Kit (ADK).

## What Was Done

### 1. Scaffolded Agent Project:

Created [ `demo-agent` ](file:///config/Desktop/BuildWithGemini/demo-agent) using:

```
`` bash
```

```
agents-cli scaffold create --adk -y demo-agent
```

```
`
```

This generates an ADK agent structure with:

- [ `app/agent.py` ](file:///config/Desktop/BuildWithGemini/demo-agent/app/agent.py) defining `root_agent` powered by `gemini-3.6-flash` with two sample function tools: `get_weather` and `get_current_time` .
- [ `app/fast_api_app.py` ](file:///config/Desktop/BuildWithGemini/demo-agent/app/fast\_api\_app.py) for serving the agent locally or in container deployments.
- Integration & unit test suites in [ `tests/` ](file:///config/Desktop/BuildWithGemini/demo-agent/tests).

### 2. Environment & Dependency Setup:

- Synchronized dependencies with `uv sync` into [ `.venv` ](file:///config/Desktop/BuildWithGemini/demo-agent/.venv).
- Configured [ `demo-agent/.env` ](file:///config/Desktop/BuildWithGemini/demo-agent/.env) with Vertex AI and your active GCP project ( `qwiklabs-gcp-04-8017249fc17a` ).

### 3. Verification:

- **Integration / Server Tests:** All 6 tests passed ( `uv run pytest` ).
- **Local Execution:** Ran an interactive query via ADK:

```
` bash
```

```
uv run adk run app "What is the weather in San Francisco and what time is it there?"
```

```
`
```

Agent output:

> In San Francisco, the weather is currently 60°F and foggy. The current time is 2:48 AM PDT.

#### How You Can Run It Locally

```
From [ /config/Desktop/BuildWithGemini/demo-agent ](file:///config/Desktop/BuildWithGemini/demo-agent):
```

- CLI prompt execution:

```
` bash
```

```
uv run adk run app "Your question here"
```

```
`
```

- Local Web UI Playground:

```
` bash
```

```
uv run adk web --port 8080 --allow_origins "*" --reload_agents
```

```
`
```

#### TURN 5 — USER

Test my agent with the message, "What's the weather in New York?"

#### TURN 6 — ASSISTANT (ANTIGRAVITY)

The agent was tested with your message. Here is the output:

```
` bash
```

```
uv run adk run app "What's the weather in New York?"
```

```
`
```

#### Response:

> [root\_agent]: The weather in New York is currently 90°F and sunny.

#### TURN 7 — USER

Launch agent playground for me.

#### TURN 8 — ASSISTANT (ANTIGRAVITY)

The ADK Agent Playground is running and launched in your browser:

- **Playground URL:** [http://localhost:8080](http://localhost:8080)
- **Status:** Active and listening on port 8080 with auto-reload enabled.

**TURN 9 — USER**

Deploy my agent to agent platform.

**TURN 10 — ASSISTANT (ANTIGRAVITY)**

I have prepared the deployment plan. Please review the [implementation plan](file:///config/.gemini/antigravity/brain/be4c90de-9c4e-4c03-a3d8-d03d0e174fd7/implementation\_plan.md).

**Summary:**

- **Target:** Google Cloud Agent Platform ( `agent_runtime` )
- **Project:** `qwiklabs-gcp-04-8017249fc17a`
- **Region:** `us-east1`
- **Test Status:** All 6 integration and unit tests passing

Deployment to Agent Runtime builds and provisions the container on Google Cloud (typically taking 5–10 minutes).

Please confirm to proceed with deploying to Agent Platform.

The deployment to Google Cloud Agent Platform is currently in progress.

- **Agent Name:** `demo-agent`
- **Resource ID:** `6669529781949693952`
- **Region:** `us-east1`
- **Operation:** `projects/384385961523/locations/us-east1/reasoningEngines/6669529781949693952/operations/8365386486220062720`
- **Cloud Console Logs:** [View Deploy Logs]  
([https://console.cloud.google.com/logs/query;query=resource.labels.reasoning\\_engine\\_id%3D%226669529781949693952%22?project=qwiklabs-gcp-04-8017249fc17a](https://console.cloud.google.com/logs/query;query=resource.labels.reasoning_engine_id%3D%226669529781949693952%22?project=qwiklabs-gcp-04-8017249fc17a))

I will monitor the deployment and notify you as soon as the agent endpoint is active and ready.

**TURN 11 — USER**

Comments on artifact URI: file:///config/.gemini/antigravity/brain/be4c90de-9c4e-4c03-a3d8-d03d0e174fd7/implementation\_plan.md

The user has approved this document.

**TURN 12 — ASSISTANT (ANTIGRAVITY)**

Deployment is continuing in the background on Agent Platform. The remote build and container provisioning are underway; I will update you with the deployed endpoint and test results as soon as it completes.

The deployment operation is still running on Google Cloud Agent Platform ( [projects/384385961523/locations/us-east1/reasoningEngines/6669529781949693952](#) ). Still monitoring.

The container build and deployment are still processing in the background (typically 5–8 minutes). Still tracking the operation.

Still deploying ( [~5 mins](#) elapsed). Continuing to monitor until completion.

Your agent has been successfully deployed to **Google Cloud Agent Platform** and verified.

See the complete [walkthrough](file:///config/.gemini/antigravity/brain/be4c90de-9c4e-4c03-a3d8-d03d0e174fd7/walkthrough.md) for full deployment parameters and endpoints.

**Deployment Summary**

- **Agent Resource:** [projects/384385961523/locations/us-east1/reasoningEngines/6669529781949693952](#)
- **Location:** [us-east1](#)
- **Service Account:** [service-384385961523@gcp-sa-aiplatform-re.iam.gserviceaccount.com](#)
- **Agent Card URL:** [agent-card.json](https://us-east1-aiplatform.googleapis.com/reasoningEngines/v1/projects/384385961523/locations/us-east1/reasoningEngines/6669529781949693952/api/a2a/app/.well-known/agent-card.json)
- **Console:** [View in Google Cloud Console](https://console.cloud.google.com/vertex-ai/agents/agent-engines/locations/us-east1/agent-engines/6669529781949693952?project=qwiklabs-gcp-04-8017249fc17a)

**Verification**

A live query was executed against the deployed Agent Runtime:

```

` bash
agents-cli run --mode a2a "What is the weather in San Francisco?"
`
Response:
> [tool_call: get_weather({"query": "San Francisco"})]
> [tool_response: get_weather -> {"result": "It's 60 degrees and foggy."}]
> The weather in San Francisco is currently 60°F and foggy.

```

**TURN 13 — USER**

Help me design my agentic application. I need help brainstorming what specific agent to build.



**TURN 14 — ASSISTANT (ANTIGRAVITY)**

To help brainstorm the best agent for you to build, let's reverse-engineer from the tools taught in this workshop.

The ideal agent fits this archetype: **a stateful conversational agent with a catalog of items it can look up/act on, remember your preferences about, and generate visuals or calculations for.**

---

**Step 1: Pick a Domain / Theme**

What domain or hobby interests you most? Here are a few high-impact directions that naturally fit the workshop tools:

**1. Travel & City Concierge** (e.g., Hidden Gems & Itinerary Planner)

- *Catalog*: Curated spots, cafes, scenic trails, local events
- *Memory*: Dietary restrictions, travel pace, favorite vibe, past trips
- *Visuals*: Photorealistic mock postcards or generated venue mood boards
- *Compute/Sandbox*: Budget tallying, travel time calculations

**2. Personal Chef & Meal Architect** (e.g., Smart Pantry & Custom Meal Prep)

- *Catalog*: Recipes, pantry ingredients, nutrition database
- *Memory*: Allergies, family member preferences, favorite cuisines, equipment owned
- *Visuals*: Plated dish previews based on custom ingredient tweaks
- *Compute/Sandbox*: Macronutrient/calorie breakdowns and portion scaling

**3. Fitness & Gear Coach** (e.g., Marathon/Triathlon or Outdoor Adventure Guide)

- *Catalog*: Workout routines, training plans, gear gear-catalog
- *Memory*: Fitness levels, target race date, injuries, heart rate zones
- *Visuals*: Form diagrams, motivational milestone badges
- *Compute/Sandbox*: Pace zone calculators, progressive overload graphs

**4. Collector / Gaming Assistant** (e.g., RPG Companion or Board Game Strategist)

- *Catalog*: Monsters/cards, game rules, character inventory
- *Memory*: Character stats, campaign history, player inventory
- *Visuals*: Character portraits, scene renders, item badges
- *Compute/Sandbox*: Combat roll calculations, probability / damage simulations

**5. Home & Garden / Specialty Shop** (e.g., Botanical Plant Care & Greenhouse)

- *Catalog*: Plants, soil types, sunlight requirements, care schedules
- *Memory*: Home lighting conditions, watering history, pets (toxicity check)
- *Visuals*: Realistic plant growth milestones or disease diagnosis mockups
- *Compute/Sandbox*: Watering frequency calculations based on humidity/temp

---

**Over to You:**

- Do any of the ideas above catch your eye?
- Or do you have a specific hobby, daily problem, or domain (e.g. coffee brewing, real estate, books, music) you'd prefer to build around?

**TURN 15 — USER**

Travel in India is fragmented: a single trip can span IRCTC, separate bus, flight, cab, and metro apps, and scattered blogs and maps for things to do, which leaves travelers facing uncertain waitlist confirmations, brittle plans that break when a train is delayed or cancelled, and generic recommendations that ignore dietary needs, pace, and vibe. This project builds an agentic Travel and City Concierge that takes a natural-language goal like "3 days in Hampi, vegetarian, slow pace, under ₹15,000" and autonomously plans, books, monitors, and re-plans the trip across train, bus, flight, car, and metro, with UPI payments that the user always approves explicitly. It tracks waitlist and RAC status, estimates confirmation probability, notifies the user on confirmation, and proposes fallbacks such as an alternate train, bus, or nearby station when confirmation looks unlikely. It builds day-wise itineraries that adapt to delays, cancellations, and weather, and acts as a city concierge that recommends hidden gems, cafes, scenic trails, and local events from a curated catalog. A memory layer stores dietary restrictions, travel pace, favorite vibes, and past trips to personalize future plans, while generated mock postcards and venue mood boards help users choose, and a compute sandbox handles budget tallying and travel-time calculations so every plan is feasible. Success is measured by the share of waitlisted trips that end in a confirmed alternative, time from request to a bookable plan, budget and preference adherence, and personalization accuracy and repeat usage, with the constraint that IRCTC lacks an open public API, so integration relies on partner APIs or approved aggregators.

**TURN 16 — ASSISTANT (ANTIGRAVITY)**

Your project brief has been saved to [ [project\\_brief.md](#) ](file:///config/Desktop/BuildWithGemini/project\_brief.md).

**Project Summary: India Travel & City Concierge (YatraAI)**

- **One-liner:** A conversational travel and city concierge that autonomously plans, adapts, and manages multi-modal India trips (trains, buses, flights, cabs) with real-time transit awareness, personalized taste profiles, and a curated catalog of hidden gems.
- **Memory:** Remembers dietary preferences (e.g., pure vegetarian, Jain), travel pace (slow, balanced), vibe preferences (heritage, architecture, nature), and previous trips.
- **Tools:** Multi-modal transit finder, waitlist & RAC fallback predictor, curated spot/cafe finder, and explicit UPI booking flow.
- **Catalog & A2UI:** Rich cards and comparison tables for day-by-day itineraries, transit legs, and curated venues.
- **Image Generation:** Custom mock postcards and visual mood boards for recommended destinations.
- **Code Sandbox:** Multi-leg budget validation and buffer calculations to ensure itineraries fit time and money constraints.

You can inspect or refine [ [project\\_brief.md](#) ](file:///config/Desktop/BuildWithGemini/project\_brief.md) as needed. When you are ready, continue with the next step of your lab guide!

**TURN 17 — USER**

Use the newly created `project_brief.md` to rename my existing agent project to match it: rename the project folder and update the name in `agents-cli-manifest.yaml` and `pyproject.toml`. Keep the code in `app/` unchanged, and don't deploy or change any agent logic yet.

**TURN 18 — ASSISTANT (ANTIGRAVITY)**

The project has been renamed to match your brief:

**1. Folder Renamed:**

- [ `/config/Desktop/BuildWithGemini/demo-agent` ](file:///config/Desktop/BuildWithGemini/demo-agent) → [ `/config/Desktop/BuildWithGemini/yatra-ai` ](file:///config/Desktop/BuildWithGemini/yatra-ai)

**2. Manifest Updated:**

- Set `name: "yatra-ai"` in [ `agents-cli-manifest.yaml` ](file:///config/Desktop/BuildWithGemini/yatra-ai/agents-cli-manifest.yaml).

**3. Packaging Updated:**

- Set `name = "yatra-ai"` in [ `pyproject.toml` ](file:///config/Desktop/BuildWithGemini/yatra-ai/pyproject.toml) and refreshed local virtualenv metadata via `uv sync`.

**4. Code Preserved:**

- All code in [ `app/` ](file:///config/Desktop/BuildWithGemini/yatra-ai/app) remains completely untouched, and no deployment or agent logic changes were made.

**TURN 19 — USER**

Give my agent a Firestore backend: a collection that fits my app (look at my `project_brief.md`) with a few sensible fields, function tools to read and write it, and a few seeded items. Important: hardcode my project ID as a string for the Firestore client and the seed script (find it with `gcloud config get-value project`). Don't read it from `google.auth.default()` or `GOOGLE_CLOUD_PROJECT`; on Agent Platform those return the project number, which breaks Firestore after you deploy.

**TURN 20 — ASSISTANT (ANTIGRAVITY)**

A Firestore backend has been integrated into your agent, tailored to your **India Travel & City Concierge (YatraAI)** brief.

---

**1. Database & Collections Setup**

- **Firestore Database:** Initialized in native mode on project `qwiklabs-gcp-04-8017249fc17a` (location `nam5`).
- **Hardcoded Project ID:** Explicitly pinned as string constant `FIRESTORE_PROJECT_ID = "qwiklabs-gcp-04-8017249fc17a"` in both the seed script and the agent code (avoiding `GOOGLE_CLOUD_PROJECT` or `google.auth.default()` which return the project number on Agent Platform).
- **Collections Created:**
  1. `curated_spots` : Catalog of hidden gems, cafes, scenic trails, and heritage sites with fields for destination, category, vibes, pace, estimated duration, cost in ₹ (INR), dietary options, and tags.
  2. `itinerary_items` : User-persisted day-wise plans and bookings with fields for destination, day number, timing, cost, notes, and category.

---

**2. Seeded Items**

Created and ran [ `seed_firestore.py` ](file:///config/Desktop/BuildWithGemini/yatra-ai/seed\_firestore.py) which populated `curated_spots` with 6 destinations:

- **Hampi:**
  - `Matanga Hill Sunrise Trek` (*scenic\_trail*, panoramic bouldering views, sunrise vibe)
  - `Mango Tree Restaurant` (*cafe*, pure vegetarian, vegan/Jain options, chill riverside vibe)
  - `Vijaya Vittala Temple & Stone Chariot` (*heritage\_site*, UNESCO monument, architecture)
- **Varanasi:**
  - `Brown Bread Bakery & Rooftop` (*cafe*, bohemian organic bakery, ghat views)
  - `Subah-e-Banaras at Assi Ghat` (*experience*, morning raga, Vedic chanting & aarti)
- **Munnar:**
  - `Chokramudi Peak Cloud Trek` (*scenic\_trail*, mist-covered shola forests & tea gardens)

---

**3. Agent Function Tools in [ `app/agent.py` ](file:///config/Desktop/BuildWithGemini/yatra-ai/app/agent.py)**

1. `search_curated_spots` : Queries Firestore with filters for `destination` , `category` , `dietary_keyword` (e.g. pure vegetarian, Jain), and `vibe` .
2. `save_user_itinerary_item` : Writes a structured itinerary item to the user's Firestore collection.
3. `get_user_itinerary` : Reads and sorts the saved itinerary items by day and time.

---

**4. Verified Locally**

Tested live via ADK:

- **Read Tool:** "Find me pure vegetarian cafes in Hampi with a chill vibe" → returned `Mango Tree Restaurant` with dietary options and price in ₹.
- **Write Tool:** "Please add Matanga Hill Sunrise Trek to Day 1 of my Hampi itinerary at 06:00 AM" → persisted to Firestore `itinerary_items` .
- **Read Saved Itinerary:** "Show my saved itinerary for Hampi" → retrieved the Day 1 Matanga Hill Trek item from Firestore.

**TURN 21 — USER**

how to check my firestore?

**TURN 22 — ASSISTANT (ANTIGRAVITY)**

You can check your Firestore database in two ways:

---

**Option 1: In the Google Cloud / Firebase Web Console (Interactive GUI)**

Click this direct link to view your data, collections, and documents in the browser:

👉 **[Open Firestore Studio in Google Cloud Console]**(<https://console.cloud.google.com/firestore/databases/default-/data/panel?project=qwiklabs-gcp-04-8017249fc17a>)

From the console, you can:

- Browse collections ( `curated_spots` , `itinerary_items` )
- View individual document fields and JSON values
- Add, edit, or delete documents manually

---

**Option 2: From the Terminal via Python CLI**

You can inspect all collections and documents directly from your project directory:

```
`` bash
|
| cd /config/Desktop/BuildWithGemini/yatra-ai
|
| uv run python -c "
|
| from google.cloud import firestore
|
| db = firestore.Client(project='qwiklabs-gcp-04-8017249fc17a')
|
| for coll in db.collections():
|
| print(f'Collection: {coll.id}')
|
| for doc in coll.stream():
|
| print(f' [{doc.id}] -> {doc.to_dict()}')
|
| "
|
|
|
|
| ##### Current Database State:
|
|
| • curated_spots (6 documents):
|
|
| • hampi-mango-tree (Mango Tree Restaurant)
|
|
| • hampi-matanga-hill (Matanga Hill Sunrise Trek)
|
|
| • hampi-vitthala-temple (Vijaya Vittala Temple & Stone Chariot)
|
|
| • varanasi-brown-bread-bakery (Brown Bread Bakery & Rooftop)
|
|
| • varanasi-morning-subah-e-banaras (Subah-e-Banaras at Assi Ghat)
|
|
```

- `munnar-chokramudi-peak` (*Chokramudi Peak Cloud Trek*)
- `itinerary_items` (1 document):
- `itin-b0941c2c`` (*Matanga Hill Sunrise Trek saved by the agent*)

#### TURN 23 — USER

Create a Cloud Storage bucket for this project. Give it a name that fits my app (look at my `project_brief.md`; add a short random suffix if the name is taken), and set the permissions so objects can be viewed publicly (i.e. embedded in a web page as images).

#### TURN 24 — ASSISTANT (ANTIGRAVITY)

A Cloud Storage bucket has been created and configured for public viewing:

##### Bucket Details

- **Bucket Name:** `yatra-ai-media-8017249fc17a`
- **URI:** `gs://yatra-ai-media-8017249fc17a`
- **Public URL Format:** `https://storage.googleapis.com/yatra-ai-media-8017249fc17a/<filename>`
- **Region:** `us-east1`
- **Project:** `qwiklabs-gcp-04-8017249fc17a`

##### Permissions & Public Access

- Granted `roles/storage.objectViewer` to `allUsers`.
- Verified live over HTTP with a test object ( `HTTP/2 200 OK` ). Images and media stored here can now be directly embedded into web pages, UI cards, and A2UI responses.

#### TURN 25 — USER

Look at my `project_brief.md` and the agent I'm building. Suggest 2-3 tools it could call to take real action or fetch real data, then recommend the simplest one to implement first.

**TURN 26 — ASSISTANT (ANTIGRAVITY)**

Based on your [ `project_brief.md` ](file:///config/Desktop/BuildWithGemini/project\_brief.md) and the current state of **YatraAI** (which already has Firestore-backed curated spot lookup and itinerary persistence), here are **3 high-impact tools** that take real action or fetch real data:

---

**1. `search_multimodal_transit(origin, destination, date, max_budget_inr)`**

- **What it does:** Searches real and alternative transit options across train, bus, and flight/cab for India corridors (e.g., Bangalore to Hampi, Delhi to Varanasi).
- **Real data/action:** Fetches schedule, duration, class (e.g. 3A, Sleeper, AC Sleeper Bus), price in ₹, and available seats.
- **Why it fits:** Directly tackles India's fragmented transit problem where rail journeys often need bus or cab connections.

---

**2. `check_waitlist_probability_and_fallbacks(train_number, pnr_or_wl_status, quota)`**

- **What it does:** Evaluates a waitlist status (e.g. `WL 24` or `RAC 12`), estimates confirmation likelihood percentage, and automatically suggests fallbacks (e.g., an alternate train, an overnight KSRTC/VRL sleeper bus, or booking from a nearby major junction).
- **Real data/action:** Returns a risk assessment score and fallback transit options if confirmation is below 60%.
- **Why it fits:** Solves the biggest anxiety in Indian rail travel—brittle plans breaking due to waitlisted tickets.

---

**3. `generate_upi_booking_intent(trip_item_id, amount_inr, payee_name, purpose)`**

- **What it does:** Takes an approved transit leg or hotel booking and constructs a standardized Indian UPI Intent URL ( `upi://pay?pa=...&am=...&pn=...&cu=INR` ) with an expiration timer and transaction ID.
- **Real data/action:** Takes a real action by producing an actionable, clickable UPI payment payload that requires explicit user confirmation before committing the booking state in Firestore.
- **Why it fits:** Perfectly matches your brief's requirement: *"autonomous planning with UPI payments that the user always approves explicitly."*

---

**💡 Recommendation: Which to implement first?**

**Start with Tool #1:** `search_multimodal_transit`

**Why it's the simplest & best first step:**

1. **Self-contained & Immediate Impact:** It gives the agent the ability to immediately answer the core query ("How do I get to Hampi from Bangalore under ₹2,000?").
2. **Natural Synergy:** It pairs directly with your existing Firestore catalog and `save_user_itinerary_item` tool. The agent can search transit, find a curated cafe/stay in Hampi, and assemble a complete Day 1 plan in a single turn.
3. **No Heavy External Dependencies:** It can start with a well-structured transit route lookup function (covering major tourist hubs like Hampi, Varanasi, Munnar, Goa, Jaipur) and later connect to live APIs or aggregator endpoints.

Would you like to implement `search_multimodal_transit` first, or would you prefer one of the others?

**TURN 27 — USER**

Implement the tool we just discussed as a function tool and add it to my agent. Keep the implementation minimal.



**TURN 28 — ASSISTANT (ANTIGRAVITY)**

The tool `search_multimodal_transit` has been implemented and added to your agent in [ `app/agent.py` ] (file:///config/Desktop/BuildWithGemini/yatra-ai/app/agent.py).

**Implementation Details:**

- **Function:** `search_multimodal_transit(origin, destination, travel_mode=None, max_budget_inr=None)`
- **Capabilities:**
  - Filters by origin and destination corridors (e.g. Bangalore to Hampi, Delhi to Varanasi, Kochi to Munnar).
  - Filters by specific mode ( `train` , `bus` , `flight` , `cab` ).
  - Filters by max budget constraint in ₹ (INR).
  - Returns structured transit details: departure/arrival stations, times, operator, duration, travel class, pricing in INR, availability, and transfer guidance.
- **Registered:** Added to `root_agent.tools` alongside `search_curated_spots` , `save_user_itinerary_item` , and `get_user_itinerary` .

**Verification:**

Tested with the query:

> "How can I travel from Bangalore to Hampi under ₹1500?"

**Agent Response:**

- Evaluated options under ₹1,500 using `search_multimodal_transit` :
1. **KSRTC Airavat AC Sleeper Bus** (₹1,150, 11:00 PM → 06:30 AM direct to Hampi Bazaar)
  2. **Hampi Express (Train 16592)** (₹890 3A, 10:00 PM SBC → 07:10 AM Hosapete + local transfer)
- Filtered out the private intercity cab (₹6,200) as it exceeded the budget cap.

**TURN 29 — USER**

Look at my `project_brief.md`. Find a free public API that fits my app's domain (browse the public-apis directory at <https://github.com/public-apis/public-apis>), then add a function tool that calls it and returns real data. Keep the implementation minimal, and if the API needs a key, read it from an environment variable rather than hardcoding it.

**TURN 30 — ASSISTANT (ANTIGRAVITY)**

A free public API from the open ecosystem—**[Open-Meteo](https://open-meteo.com/)** (listed in the public-apis directory)—has been integrated into your agent as a function tool.

**Tool:** `get_live_destination_weather`

- **API Used:** Open-Meteo Geocoding & Forecast APIs (free, no API key required, highly accurate for Indian coordinates and microclimates).
- **Location in Code:** Added to [ `app/agent.py` ](file:///config/Desktop/BuildWithGemini/yatra-ai/app/agent.py).
- **What it does:**

1. Dynamically geocodes any Indian city, town, or heritage site (e.g., `Hampi` , `Varanasi` , `Munnar` , `Goa` ).
2. Fetches real-time temperature (°C), weather conditions (interpreting WMO weather codes), humidity (%), precipitation (mm), and wind speed.
3. Returns contextual travel and outdoor trekking advice based on current weather conditions.

- **Registered:** Included in `root_agent.tools` .

**Verification Test**

Tested live through the agent via `adk run` :

> "What is the live weather in Hampi right now and is it good for a trek?"

**Live Agent Output:**

- Triggered `get_live_destination_weather(destination="Hampi")` .
- Retrieved live data: **32°C, partly cloudy/overcast, 53% humidity, 0.0 mm precipitation.**
- Provided trekking guidance tailored to Hampi's boulder terrain and midday temperatures.

**TURN 31 — USER**

First, enable the Geocoding and Places (New) APIs on my project. Here's my Maps

API key: PASTE\_KEY\_HERE. Save it as GOOGLE\_MAPS\_API\_KEY in my local .env; don't

hardcode it in a source file. Then add a tool that uses the

Geocoding API to turn an address into coordinates, and one that uses the Places

API (New) to find nearby places of a given type. Call the REST endpoints

directly and return the key fields (name, address, location). Use the Developer

Knowledge MCP to confirm the exact endpoints and required headers.

**TURN 32 — ASSISTANT (ANTIGRAVITY)**

The Maps APIs have been enabled, your `.env` configuration prepared, and the two function tools implemented and registered with your agent.

---

**1. APIs Enabled**

Enabled on project `qwiklabs-gcp-04-8017249fc17a` :

- **Geocoding API:** `geocoding-backend.googleapis.com`
- **Places API (New):** `places.googleapis.com`

---

**2. Environment Configuration**

Added the variable to `[ yatra-ai/.env ]`(file:///config/Desktop/BuildWithGemini/yatra-ai/.env):

```
`` bash
```

```
GOOGLE_MAPS_API_KEY=PASTE_KEY_HERE
```

```
`
```

> **Next Step for you:** Replace `PASTE_KEY_HERE` with your actual Maps API key in `[ yatra-ai/.env ]` (file:///config/Desktop/BuildWithGemini/yatra-ai/.env). The code reads `os.environ.get("GOOGLE_MAPS_API_KEY")` and does not hardcode the key in any source file.

---

**3. Endpoints & Headers (Confirmed via Developer Knowledge MCP)**

- **Geocoding API:**

- **Endpoint:** GET `https://maps.googleapis.com/maps/api/geocode/json?address={address}&key={api_key}`

- **Header:** `X-Goog-API-Key`

- **Output:** Returns formatted address and lat/lng coordinates.

- **Places API (New) Nearby Search:**

- **Endpoint:** POST `https://places.googleapis.com/v1/places:searchNearby`

- **Headers:**

- `X-Goog-API-Key: <api-key>`

- `X-Goog-FieldMask: places.displayName,places.formattedAddress,places.location,places.primaryType`

- `Content-Type: application/json`

- **Payload:** Circle locationRestriction with center coordinates and radius , along with includedTypes (e.g., tourist\_attraction , restaurant , cafe ).

#### 4. Function Tools Added in [ app/agent.py ](file:///config/Desktop/BuildWithGemini/yatra-ai/app/agent.py)

##### 1. geocode\_address(address: str) :

- Calls the Geocoding REST endpoint directly.
- Extracts formatted\_address , location.latitude , location.longitude , and place\_id .

##### 2. find\_nearby\_places(latitude: float, longitude: float, place\_type: str, radius\_meters: float, max\_results: int) :

- Calls the Places API (New) searchNearby REST endpoint.
- Returns key fields: name , address , location , and primary\_type .

##### 3. Registered: Both tools are registered in root\_agent.tools`.

### TURN 33 — USER

Add a tool that generates an image for an item in my agent's domain (look at my project\_brief.md) using the gemini-3.1-flash-lite-image model in the global region. Do two things with the generated image: (1) save it with tool\_context.save\_artifact so it shows up in the Playground's Artifacts panel, and (2) upload the same image bytes to the public Cloud Storage bucket I created earlier and return its public https URL (<https://storage.googleapis.com/<bucket>/<object>>) from the tool. Hardcode the bucket name as a string, the same way we hardcoded the Firestore project. Do not write the image to a local file and return a path. Use the Developer Knowledge MCP to confirm the API if you're unsure.

**TURN 34 — ASSISTANT (ANTIGRAVITY)**

The image generation tool `generate_destination_postcard` has been implemented and added to your agent in `[app/agent.py]` (`file:///config/Desktop/BuildWithGemini/yatra-ai/app/agent.py`).

**How it works:**

1. **Model & Region:** Uses `gemini-3.1-flash-lite-image` in the `global` region via `google.genai.Client(vertexai=True, project="qwiklabs-gcp-04-8017249fc17a", location="global")` with `response_modalities=["IMAGE"]`.
2. **Artifacts Panel:** Calls `await tool_context.save_artifact(filename=artifact_filename, artifact=types.Part.from_bytes(...))` so the generated visual immediately registers in the Playground's Artifacts sidebar.
3. **Public Cloud Storage:** Uploads the exact image bytes in memory directly to your public bucket `yatra-ai-media-8017249fc17a` (no local files written).
4. **Public HTTPS Return:** Returns the direct public URL `https://storage.googleapis.com/yatra-ai-media-8017249fc17a/postcards/<filename>` which renders inline in chat and web UIs.

**Verification:**

Tested with the query:

> *"Generate a postcard for Hampi showing the stone chariot at golden sunrise."*

- **Result:** Successfully generated the image, saved it as an artifact, uploaded it to GCS, and returned `https://storage.googleapis.com/yatra-ai-media-8017249fc17a/postcards/postcard_hampi_ce39ce.jpg` ( `HTTP/2 200 OK` ).

**TURN 35 — USER**

Add Agent Platform code execution to my agent using `AgentEngineSandboxCodeExecutor` so it can safely run Python in a sandbox. If I don't already have a sandbox, create one from the Agent Engine in my `deployment_metadata.json`.

**TURN 36 — ASSISTANT (ANTIGRAVITY)**

Waiting for the Agent Engine code execution sandbox to finish provisioning...

Still provisioning the sandbox environment on Agent Engine ( ~40s elapsed). Monitoring...

Still waiting on the sandbox provisioning operation...

Checking the execution result from the sandbox code executor...

Agent Platform code execution has been added to your agent using `AgentEngineSandboxCodeExecutor` !

**What was done:**

1. **Agent Engine ID:** Read the deployed Agent Engine runtime resource from [ `deployment_metadata.json` ] (file:///config/Desktop/BuildWithGemini/yatra-ai/deployment\_metadata.json):

```
``
|
|
| projects/384385961523/locations/us-east1/reasoningEngines/6669529781949693952
|
|
| ``
```

**2. Sandbox Provisioning:**

Checked for existing sandboxes on this Agent Engine; since none existed, created a dedicated code execution sandbox environment:

```
|
| `
|
| projects/384385961523/locations/us-
| east1/reasoningEngines/6669529781949693952/sandboxEnvironments/7992983341001342976
|
| `
|
|
```

- **State:** `STATE_RUNNING`

- **Display Name:** `yatra_ai_sandbox`

- **TTL:** `1 year ( 31536000s )`

3. **Agent Configuration** in [ `app/agent.py` ] (file:///config/Desktop/BuildWithGemini/yatra-ai/app/agent.py):

`Configured` `AgentEngineSandboxCodeExecutor` `directly on` `root_agent` :

```
|
| ` python
```

```
|
| code_executor=AgentEngineSandboxCodeExecutor(
|
| sandbox_resource_name="projects/384385961523/locations/us-
| east1/reasoningEngines/6669529781949693952/sandboxEnvironments/7992983341001342976",
| agent_engine_resource_name="projects/384385961523/locations/us-east1/reasoningEngines/6669529781949693952",
| )
```

#### 4. Live Verification:

Executed a live calculation query through `adk run`. The agent generated the Python code block and triggered the sandbox execution endpoint:

```
POST https://us-east1-aiplatform.googleapis.com/v1beta1/projects/384385961523/locations/us-east1/reasoningEngines/6669529781949693952/sandboxEnvironments/7992983341001342976/:execute -> HTTP/1.1 200 OK
```

The sandbox executed the code and returned the results safely.

#### TURN 37 — USER

Use the memory-bank-setup skill to add memory to my agent. Reuse the Agent Engine from my earlier deployment as the Memory Bank (its ID is the last part of `remote_agent_runtime_id` in `deployment_metadata.json`). Also set the memory service in the app code so it's used when I redeploy later, but don't redeploy now. Then restart my local playground with `uv run adk web . --port 8080 --reload_agents --memory_service_uri=agentengine://<ID>` instead of `agents-cli playground`, and use that same command whenever I ask you to launch or restart the playground.

**TURN 38 — ASSISTANT (ANTIGRAVITY)**

Long-term cross-session memory has been added to your agent using the `memory-bank-setup` skill, and your local playground is running with the requested memory configuration.

---

**1. Agent Engine Reused as Memory Bank**

- **Memory Bank ID:** `6669529781949693952` (extracted from `remote_agent_runtime_id` in `[ deployment_metadata.json ]` (file:///config/Desktop/BuildWithGemini/yatra-ai/deployment\_metadata.json)).
- **Resource URI:** `agentengine:///6669529781949693952`

---

**2. Agent Code Wiring in [ `app/agent.py` ](file:///config/Desktop/BuildWithGemini/yatra-ai/app/agent.py)****1. Memory Callback ( `WRITE` ):**

```
``python
|
|
| async def generate_memories_callback(callback_context: CallbackContext):
|
|     """After each turn, send session events to Memory Bank for extraction."""
|
|     await callback_context.add_session_to_memory()
|
|     return None
|
| `
```

**2. PreloadMemoryTool ( `READ` ) :**

Added `PreloadMemoryTool()` into `root_agent.tools` so durable memories/preferences are fetched and injected into the prompt at the start of each turn.

3. **Registered:** Set `after_agent_callback=generate_memories_callback` on `root_agent` .

---

**3. App Service Configuration for Future Redeployments**

To ensure memory persists when redeployed later to Agent Runtime (which defaults to in-memory unless explicitly provided):

- [ `app/app_utils/services.py` ](file:///config/Desktop/BuildWithGemini/yatra-ai/app/app\_utils/services.py) : Added `get_memory_service()` returning `VertexAiMemoryBankService(project="qwiklabs-gcp-04-8017249fc17a", location="us-east1", agent_engine_id="6669529781949693952")` , registered under `shared://memory` .
- [ `app/fast_api_app.py` ](file:///config/Desktop/BuildWithGemini/yatra-ai/app/fast\_api\_app.py) : Attached `memory_service=services.get_memory_service()` to the A2A Runner and passed `memory_service_uri=services.MEMORY_SERVICE_URI` to `get_fast_api_app` .

---

**4. Local Playground Restarted**



Started as a background process with:

```
` bash
```

```
uv run adk web . --port 8080 --reload_agents --memory_service_uri=agentengine://6669529781949693952
```

```
`
```

- **URL:** [http://localhost:8080](http://localhost:8080)
- This exact command will be used whenever you request to launch or restart the playground.

#### TURN 39 — USER

Depending on your application, you can be more specific about what to remember:

Example: "Configure memory so that all user allergies are remembered." alter this according to your application

#### TURN 40 — ASSISTANT (ANTIGRAVITY)

Your agent's Memory Bank configuration and system prompt have been tailored to **YatraAI's India Travel & City Concierge** domain.

**Configured Memory Profile Dimensions (in [ `app/agent.py` ](file:///config/Desktop/BuildWithGemini/yatra-ai/app/agent.py)):**

##### 1. Dietary Restrictions & Allergies:

- Strict vegetarian, Jain (strictly no onion, garlic, or root vegetables), vegan, halal, gluten-free, peanut or lactose allergies.
- Automatically filters curated restaurant & cafe recommendations without needing to re-prompt the user each session.

##### 2. Travel Pace & Rhythm:

- Pacing style: *slow & leisurely* (max 2–3 spots/day, late relaxed mornings), *balanced*, or *fast-paced sightseeing sprint*.

##### 3. Vibe & Experience Preferences:

- Heritage & boulder trails, sunset viewpoints, riverside cafe culture, spiritual temples, or photography spots.

##### 4. Origin & Transit Preferences:

- Origin city (e.g. Bangalore, Mumbai, Hyderabad), preferred train coach classes (Vande Bharat, 1A/2A/3A, AC Sleeper bus vs private cab).

##### 5. Budget Tier:

- Backpacker budget (under ₹5,000), comfortable mid-tier (under ₹15,000), or premium heritage stays.

##### 6. Past Trips & Visited Spots:

- Locations already explored or liked/disliked to avoid redundant recommendations.

The local playground daemon running at [http://localhost:8080](http://localhost:8080) automatically reloads agents with `--reload_agents` and persists extracted user facts directly into your Vertex AI Memory Bank (`agentengine://6669529781949693952`).

#### TURN 41 — USER

Use the enable-a2ui skill to add A2UI to my agent: build the system prompt with A2uiSchemaManager (version 0.8) and the Basic Catalog, copy in a2ui\_utils.py, and wire it up as an after\_model\_callback.

**TURN 42 — ASSISTANT (ANTIGRAVITY)**

A2UI has been added to your agent using the `enable-a2ui` skill!

**Summary of Changes:****1. A2UI SDK:**

- Installed `a2ui-agent-sdk` into the project environment.

**2. Template Callback Copied:**

- Copied the ready-made `[a2ui_utils.py]` (file:///config/Desktop/BuildWithGemini/yatra-ai/app/a2ui\_utils.py) into `app/a2ui_utils.py` to unwrap and rewrap A2UI v0.8 payloads (`beginRendering`, `surfaceUpdate`, data models) into `<a2a_datapart_json>` format for `adk web`.

**3. System Prompt with `A2uiSchemaManager` (v0.8):**

- Configured `A2uiSchemaManager(version="0.8", catalogs=[BasicCatalog.get_config("0.8")])` in `[app/agent.py]` (file:///config/Desktop/BuildWithGemini/yatra-ai/app/agent.py).
- Generated the system instruction containing the full A2UI v0.8 schema, flat card/column/row component constraints, public HTTPS image handling, and few-shot examples.

**4. Agent Wiring:**

- Configured `after_model_callback=a2ui_callback` on `root_agent`.
- Retained the `after_agent_callback=generate_memories_callback` for Vertex AI Memory Bank and `PreloadMemoryTool()`.

**5. Hot-Reload Active:**

- The local playground (`uv run adk web . --port 8080 --reload_agents --memory_service_uri=...`) detected the changes in `app/agent.py` and `app/a2ui_utils.py` and reloaded the agent live.

> **Tip for testing in the Playground** (`http://localhost:8080`):

> Turn **Token Streaming OFF** (gear icon in the playground header) and start a **New Session** so `adk web` can swap the raw JSON with rich cards.

**TURN 43 — USER**

Restart the agent playground so it picks up my latest changes.

**TURN 44 — ASSISTANT (ANTIGRAVITY)**

The agent playground has been restarted with your latest changes:

```
`` bash
|
| uv run adk web . --port 8080 --reload_agents --memory_service_uri=agentengine:///6669529781949693952
|
| ``
```

- Playground URL:** `[http://localhost:8080](http://localhost:8080)`
- Status:** Running and listening on port 8080.
- Tip:** Remember to turn **Token Streaming OFF** in the playground settings (gear icon) and start a **New Session** to render the A2UI cards.

**TURN 45 — USER**

Redeploy my agent to Agent Platform. If my agent reads any API keys from environment variables (like `GOOGLE_MAPS_API_KEY`), pass them to the deployment with `--update-env-vars`.

**TURN 46 — ASSISTANT (ANTIGRAVITY)**

The agent is being redeployed to Agent Platform. Monitoring the deployment progress in the background...

Still building the container and updating Agent Runtime on Google Cloud (1m 21s elapsed). Monitoring in the background...

Redeployment has been submitted to Agent Platform with `GOOGLE_MAPS_API_KEY` included. Monitoring the cloud container build and rollout in the background...

Still building container and provisioning Agent Runtime (1m 16s elapsed). Monitoring in the background...

Still deploying (2m 27s elapsed). Monitoring in the background...

Your agent has been redeployed to Agent Platform!

**Deployment Details:**

- **Agent Runtime ID:** `projects/384385961523/locations/us-east1/reasoningEngines/7039950851300917248`
- **Region:** `us-east1`
- **Service Account:** `service-384385961523@gcp-sa-aipatform-re.iam.gserviceaccount.com`
- **Agent Card URL:** `[https://us-east1-aipatform.googleapis.com/reasoningEngines/v1/projects/384385961523/locations/us-east1/reasoningEngines/7039950851300917248/api/a2a/app/.well-known/agent-card.json](https://us-east1-aipatform.googleapis.com/reasoningEngines/v1/projects/384385961523/locations/us-east1/reasoningEngines/7039950851300917248/api/a2a/app/.well-known/agent-card.json)`
- **Console Link:** [View Agent in Vertex AI Console](https://console.cloud.google.com/vertex-ai/agents/agent-engines/locations/us-east1/agent-engines/7039950851300917248?project=qwiklabs-gcp-04-8017249fc17a)

**Environment & Integrations:**

- Passed `GOOGLE_MAPS_API_KEY` via `--update-env-vars`.
- Package dependencies (including `a2ui-agent-sdk`, `google-cloud-firestore`) built into the container.
- Persistent Vertex AI Memory Bank and code execution sandbox wired in.
- Updated `[ deployment_metadata.json ]`(file:///config/Desktop/BuildWithGemini/yatra-ai/deployment\_metadata.json) with the new runtime ID.

**TURN 47 — USER**

Grant my deployed agent's service account the roles it needs: roles/datastore.user for Firestore, and roles/storage.objectAdmin on my image bucket if it generates images.

**TURN 48 — ASSISTANT (ANTIGRAVITY)**

The requested IAM roles have been granted to your deployed agent's service account ( `service-384385961523@gcp-sa-aiplatform-re.iam.gserviceaccount.com` ):

**1. Firestore Access:**

- Role: `roles/datastore.user`
- Scope: Project-level ( `qwiklabs-gcp-04-8017249fc17a` )

**2. Cloud Storage Image Bucket Access:**

- Role: `roles/storage.objectAdmin`
- Scope: Bucket-level ( `gs://yatra-ai-media-8017249fc17a` )

**TURN 49 — USER**

Grant my deployed agent's service account the roles it needs: `roles/datastore.user` for Firestore, and `roles/storage.objectAdmin` on my image bucket if it generates images.

**TURN 50 — ASSISTANT (ANTIGRAVITY)**

Both permissions are already granted and active on your deployed agent's service account ( `service-384385961523@gcp-sa-aiplatform-re.iam.gserviceaccount.com` ):

**1. Firestore Access:**

- Role: `roles/datastore.user`
- Scope: Project `qwiklabs-gcp-04-8017249fc17a`
- Verified active: `service-384385961523@gcp-sa-aiplatform-re.iam.gserviceaccount.com` has `roles/datastore.user` in the project IAM policy.

**2. Cloud Storage Image Bucket Access:**

- Role: `roles/storage.objectAdmin`
- Scope: Bucket `gs://yatra-ai-media-8017249fc17a`
- Verified active: Explicitly bound in the bucket IAM policy with full object administrative access.

**TURN 51 — USER**

Using the `build-agent-frontend` skill, copy its minimal FastAPI proxy and chat UI template into `./frontend` and wire it to my deployed agent using `AGENT_ENGINE_RESOURCE_NAME` and `AGENT_DIRECTORY` (my `agent_directory` from `agents-cli-manifest.yaml`). Keep it a plain chat UI. Do not build a React app or pull in a large sample frontend.

**TURN 52 — ASSISTANT (ANTIGRAVITY)**

The minimal FastAPI proxy and plain chat UI template from the `build-agent-frontend` skill have been set up in `[ frontend/ ]` (file:///config/Desktop/BuildWithGemini/yatra-ai/frontend/):

**1. Structure Copied**

- **Proxy:** `[ frontend/main.py ]`(file:///config/Desktop/BuildWithGemini/yatra-ai/frontend/main.py)
- **Dependencies:** `[ frontend/requirements.txt ]`(file:///config/Desktop/BuildWithGemini/yatra-ai/frontend/requirements.txt)
- **Chat UI & Built-in A2UI Renderer:** `[ frontend/static/index.html ]`(file:///config/Desktop/BuildWithGemini/yatra-ai/frontend/static/index.html)

**2. Environment & Routing Configuration**

- Wired to your deployed agent via `[ frontend/.env ]`(file:///config/Desktop/BuildWithGemini/yatra-ai/frontend/.env):

```
`` bash
AGENT_ENGINE_RESOURCE_NAME="projects/384385961523/locations/us-east1/reasoningEngines/7039950851300917248"
AGENT_DIRECTORY="app"
```

- Verified that the A2A endpoint responds with your deployed agent card containing all tools, memory, code execution, and A2UI schema.

**3. How to Run Locally**

Whenever you want to run the frontend proxy locally:

```
`` bash
cd /config/Desktop/BuildWithGemini/yatra-ai/frontend
./venv/bin/python main.py
```

This serves the chat interface on `http://localhost:8080``, handling plain text responses as well as native A2UI cards.

**TURN 53 — USER**

Run my frontend locally from the frontend/ folder: install its dependencies, set `AGENT_ENGINE_RESOURCE_NAME` to the resource name in `deployment_metadata.json` and `AGENT_DIRECTORY` to my `agent_directory` from `agents-cli-manifest.yaml`, then start the server on `http://localhost:8080`.

**TURN 54 — ASSISTANT (ANTIGRAVITY)**

Your frontend server is installed, configured, and running locally!

**Setup Summary:**

- **Dependencies:** Installed from [ `frontend/requirements.txt` ](file:///config/Desktop/BuildWithGemini/yatra-ai/frontend/requirements.txt) into `frontend/.venv`.
- **Environment Variables:**
  - `AGENT_ENGINE_RESOURCE_NAME="projects/384385961523/locations/us-east1/reasoningEngines/7039950851300917248"`
  - `AGENT_DIRECTORY="app"`
  - `PORT=8080`
- **Server URL:** [http://localhost:8080](http://localhost:8080)
- **Status:** Running in the background ( `Uvicorn running on http://0.0.0.0:8080` ). You can open [http://localhost:8080](http://localhost:8080) in your browser to chat with your deployed agent and view plain text responses as well as native A2UI cards.

**TURN 55 — USER**

Deploy the frontend to Cloud Run pointing at my `AGENT_ENGINE_RESOURCE_NAME` and `AGENT_DIRECTORY`, and grant the Cloud Run service account roles/aiplatform.user so it can reach the agent.

**TURN 56 — ASSISTANT (ANTIGRAVITY)**

The frontend is being built and deployed to Cloud Run with `roles/aiplatform.user` granted to its service account. The process will complete in the background.

The frontend has been deployed to Cloud Run and is wired to your deployed agent!

**Deployment Summary:**

- **Cloud Run Service:** `yatra-ai-frontend`
- **Region:** `us-east1`
- **Service URL:** [https://yatra-ai-frontend-384385961523.us-east1.run.app](https://yatra-ai-frontend-384385961523.us-east1.run.app)
- **Status:** Live (HTTP 200 OK)

**Configuration & Access:**

- **Environment Variables:**
  - `AGENT_ENGINE_RESOURCE_NAME="projects/384385961523/locations/us-east1/reasoningEngines/7039950851300917248"`
  - `AGENT_DIRECTORY="app"`
- **IAM Permission Granted:**
  - Bound `roles/aiplatform.user` on the Cloud Run default service account ( `384385961523-compute@developer.gserviceaccount.com` ), allowing the proxy to communicate with your deployed Agent Runtime instance over the A2A protocol.

**TURN 57 — USER**

Rebrand my frontend: set the title and header to my app's name and change the accent color.

**TURN 58 — ASSISTANT (ANTIGRAVITY)**

The frontend branding has been updated:

**Brand Updates:**

- **Page Title:** YatraAI – Travel & City Concierge
- **Header:** YatraAI with subtitle *India Travel & City Concierge*
- **Accent Theme:** Warm terracotta/saffron ( #E65100 ) with matching cream/neutral surfaces ( #FBF8F5 , #E8DDD3 ), fitting the Indian travel concierge domain.

**Availability:**

- **Local Dev Server:** Immediately visible on [http://localhost:8080](http://localhost:8080).
- **Cloud Run Service:** Revision deployment is building and rolling out to [https://yatra-ai-frontend-384385961523.us-east1.run.app](https://yatra-ai-frontend-384385961523.us-east1.run.app) in the background.

**TURN 59 — USER**

Add a row of 3 clickable example prompts above the input, tailored to my agent (look at my project\_brief.md).

**TURN 60 — ASSISTANT (ANTIGRAVITY)**

A row of 3 clickable example prompt chips has been added directly above the input form, tailored to your agent's domain from `project_brief.md`:

**Added Example Chips:**1. 🏛️ **3 Days in Hampi (Veg, ₹15k):**

- **Prompt:** "Plan 3 days in Hampi, vegetarian, slow pace, under ₹15,000"
- Exercises itinerary planning, dietary awareness, pace constraints, and budget tallying.

2. 🚆 **Bangalore to Hosapete Transit:**

- **Prompt:** "Find trains and sleeper buses from Bangalore to Hosapete under ₹2,000"
- Exercises multi-modal transit search across Indian Railways and buses with pricing.

3. 🌅 **Generate Hampi Postcard:**

- **Prompt:** "Generate a sunset travel postcard for Virupaksha Temple in Hampi"
- Exercises Imagen visual postcard generation and public Cloud Storage image display in A2UI.

**Behavior:**

- Clicking any chip automatically dispatches the message into the chat flow and shows the agent's stream and cards.
- **Local Dev Server:** Active right now on [http://localhost:8080](http://localhost:8080).
- **Cloud Run:** Redeployment with the new chips is updating in the background at [https://yatra-ai-frontend-384385961523.us-east1.run.app](https://yatra-ai-frontend-384385961523.us-east1.run.app).

**TURN 61 — USER**

Create a nice dialogue layout for my app that matches my theme.

**TURN 62 — ASSISTANT (ANTIGRAVITY)**

A dialogue layout matching your India Travel & City Concierge theme has been crafted and deployed:

**Dialogue Layout Features:****1. Avatar-Anchored Dialogue Rows:**

- **User:** Deep saffron bubble ( #E65100 ) right-aligned with user badge and subtle glow.
- **YatraAI:** Clean elevated card surface left-aligned with a dedicated 🚐 concierge avatar and sender identity.

**2. Warm Destination Concierge Aesthetics:**

- Header with gradient terracotta ( #E65100 to #BF360C ) and an A2A Concierge pill badge.
- Centered greeting card: *"Namaste! Where would you like to travel?"* welcoming the user with suggested capabilities.
- Warm sandstone and spice palette ( #FBF8F5 background, #E8DDD3 borders, #6D4C41 secondary accents).

**3. Refined Controls & A2UI Cards:**

- Floating interactive quick-prompt chips with hover lift effects.
- Rounded input bar with send button.
- Polished styling for A2UI cards, dividers, and images.

**Availability:**

- **Local Dev Server:** Live now at [http://localhost:8080](http://localhost:8080).
- **Cloud Run:** Updating revision in the background at [https://yatra-ai-frontend-384385961523.us-east1.run.app](https://yatra-ai-frontend-384385961523.us-east1.run.app).

The Cloud Run service update is progressing smoothly and will complete shortly. Both your local server at [http://localhost:8080](http://localhost:8080) and your public Cloud Run endpoint at [https://yatra-ai-frontend-384385961523.us-east1.run.app](https://yatra-ai-frontend-384385961523.us-east1.run.app) are equipped with the themed dialogue layout, quick-action chips, and A2A backend wiring.

**TURN 63 — USER**

Add a tool that generates a short video for an item in my agent's domain (look at my project\_brief.md) using Google's Omni model (gemini-omni-flash-preview) in the global region. Do two things with the generated video: (1) save it with tool\_context.save\_artifact so it shows up in the Playground's Artifacts panel, and (2) upload the same video bytes to the public Cloud Storage bucket I created earlier and return its public https URL (https://storage.googleapis.com/<bucket>/<object>) from the tool. Hardcode the bucket name as a string, the same way we hardcoded the Firestore project. Do not write the video to a local file and return a path. Use the Developer Knowledge MCP to confirm the API if you're unsure.



**TURN 64 — ASSISTANT (ANTIGRAVITY)**

I am testing `gemini-omni-flash-preview` in the `global` region with the Google GenAI SDK to verify video generation before wiring the tool to your agent. I will update you as soon as the test finishes.

The video generation call to `gemini-omni-flash-preview` is processing in the background. I'll inspect the generated video output once it returns.

The video generation request is still running on Vertex AI (`gemini-omni-flash-preview`). I'll inspect the response as soon as it completes.

Still generating the video with `gemini-omni-flash-preview` (video rendering usually takes ~1-2 minutes). I will check back shortly.

The video generation tool has been added to your agent:

**Tool Details:**

- **Function:** `generate_destination_video(destination, scene_description, tool_context)` in `[ app/agent.py ]` (file:///config/Desktop/BuildWithGemini/yatra-ai/app/agent.py)
- **Model:** Google's Omni model `gemini-omni-flash-preview` in the `global` region via `client.interactions.create()`.
- **Artifact Saving:**
  - Encodes the video bytes into `types.Part.from_bytes(data=video_bytes, mime_type="video/mp4")`.
  - Saves the artifact with `await tool_context.save_artifact(...)` so it appears in the Playground's Artifacts panel.
- **Cloud Storage Upload:**
  - Hardcoded bucket: `"yatra-ai-media-8017249fc17a"`.
  - Uploads the video bytes directly via `storage.Client(project="qwiklabs-gcp-04-8017249fc17a")` under `videos/<filename>.mp4`.
  - Returns the public URL (`https://storage.googleapis.com/yatra-ai-media-8017249fc17a/videos/<filename>.mp4`) without writing to any temporary local file path.
- **Agent Integration:** Registered on `root_agent.tools`.

**TURN 65 — USER**

Record a demo video of my agent. Show it doing the thing my app does best, then ask a second, richer prompt that shows off a tool call, a database lookup, or a generated image.

**TURN 66 — ASSISTANT (ANTIGRAVITY)**

The query is running through the agent and looking up curated spots in Firestore over A2A. Once it returns, I will record the full video walkthrough.

The demo recording script is executing in headless Chromium with high-definition capture. It is submitting the trip planner query and then following up with the curated database lookup and image generation prompt. I will notify you with the final video as soon as encoding finishes.

Prompt 1 completed with the 3-day itinerary. Prompt 2 (curated spots lookup & postcard image) is processing and will finalize the video shortly.

I have recorded a demo video of your agent showcasing its full range of capabilities:

**Demo Highlights in the Video:****1. First Interaction (Core Strength):**

- **Prompt:** "Plan 3 days in Hampi, vegetarian, slow pace, under ₹15,000"
- **Result:** The agent delivers a paced, day-by-day travel itinerary with realistic timings, pure vegetarian dining highlights (like Mango Tree restaurant), local transport breakdowns, and a ₹14,000 budget summary inside an A2UI card.

**2. Second Interaction (Multi-Tool Call & Rich Output):**

- **Prompt:** "Show me curated vegetarian hidden gems in Hampi from your catalog, and generate a sunset postcard."
- **Result:** The agent executes a Firestore database query ( `search_curated_spots` ) for vegetarian heritage spots and invokes `generate_destination_postcard` via `gemini-3.1-flash-lite-image` , rendering a sunset view of Hampi's Virupaksha temple and boulder ruins directly in the chat dialogue.

**Artifacts Created:**

- **Video file:** [ `demo_video.mp4` ](file:///config/Desktop/BuildWithGemini/yatra-ai/demo\_video.mp4) (H.264 MP4, 1280x800, ~57 seconds)
- **High-res screenshot:** [ `demo_screenshot.png` ](file:///config/Desktop/BuildWithGemini/yatra-ai/demo\_screenshot.png)

**TURN 67 — USER**

Generate a README.md for my project. Describe what my agent actually does based on the code in this repo — read `app/` and `agents-cli-manifest.yaml` to see which tools and Google Cloud services are really wired up (Memory Bank, Firestore, Cloud Storage, image generation, A2UI) and list ONLY those. Don't claim a capability the code doesn't implement; if something in my `project_brief.md` was planned but not finished, leave it out or mark it clearly as "planned, not yet implemented." Do NOT put any live links to localhost (like `http://localhost:8080`) or to ephemeral Cloud URLs (Cloud Run / Agent Engine endpoints from this lab) in the README — they die when this workstation is torn down and will 404 for anyone reading it later. If you need to show how to run it, write it as setup/run instructions (the commands to start it locally), not as a clickable link. Then convert the demo I recorded with `record-demo` (the `.webm` file) to an optimized, looping GIF and embed it near the top with a relative path so it plays inline — use only that real recording; if there isn't one, leave the demo out rather than generating or drawing a placeholder.

**TURN 68 — ASSISTANT (ANTIGRAVITY)**

A [ `README.md` ](file:///config/Desktop/BuildWithGemini/yatra-ai/README.md) has been generated for your repository:

**What was completed:****1. Looping Demo GIF:**

- Converted the recorded session ( `page@c437bfa8e91264d3ac1483c3f356c84b.webm` ) into an optimized, looping [ `demo.gif` ] (file:///config/Desktop/BuildWithGemini/yatra-ai/demo.gif) using a custom two-pass FFmpeg palette.
- Embedded at the top of the README using a clean relative link ( `![YatraAI Demo](demo.gif)` ).

**2. Accurate Capabilities Inventory:**

- Lists only the tools and Google Cloud integrations implemented in [ `app/agent.py` ] (file:///config/Desktop/BuildWithGemini/yatra-ai/app/agent.py) and [ `agents-cli-manifest.yaml` ] (file:///config/Desktop/BuildWithGemini/yatra-ai/agents-cli-manifest.yaml):
- **Vertex AI Memory Bank:** Cross-session user preferences via `PreloadMemoryTool` and `add_session_to_memory()` .
- **Google Cloud Firestore:** Curated catalog filtering ( `search_curated_spots` ) and trip itinerary persistence ( `save_user_itinerary_item` , `get_user_itinerary` ).
- **Cloud Storage & Generative Media:** Postcards ( `gemini-3.1-flash-lite-image` ) and cinematic video ( `gemini-omni-flash-preview` ), uploaded to Cloud Storage and saved to Playground Artifacts.
- **Transit, Weather, & Places:** Indian multi-modal transit search, live Open-Meteo weather alerts, and Google Maps geocoding / nearby places.
- **Python Code Execution Sandbox:** Vertex AI Agent Engine sandbox for travel mathematics.
- **A2UI v0.8 Rich UI:** Card, Column, Row, Text, and Image component rendering over A2A.

**3. Production Run & Setup Instructions:**

- Outlines local dependency installation with `uv sync` .
- Explains how to run the ADK web development UI with Memory Bank ( `uv run adk web . --memory_service_uri=...` ).
- Explains how to launch the FastAPI A2A frontend proxy ( `python main.py` ).
- Excludes ephemeral localhost/cloud URLs to ensure longevity after workstations are torn down.

**TURN 69 — USER**

Publish my project to GitHub and submit it for swag.

**TURN 70 — ASSISTANT (ANTIGRAVITY)**

Please authenticate with GitHub to publish your project:

1. Open [<https://github.com/login/device>](<https://github.com/login/device>) in your browser.
2. Sign in with **your personal GitHub account** (not the lab/Qwiklabs account).
3. Enter this one-time code:

**C1E6-3621**

Once you have authorized the device code, let me know or confirm here, and we'll review the repository name and push your code.

You are authenticated as **Jeevanthikashyap**.

I am ready to create the **public** GitHub repository:

- **Repository Name:** **buildwithgemini-yatra-ai**
- **GitHub Account:** **Jeevanthikashyap**
- **Visibility: Public** (required for the showcase gallery and swag redemption)

Are you good to proceed with creating this repository and pushing your code?

**TURN 71 — USER**

but how can i view the repo where you have pushed the whole project?

**TURN 72 — ASSISTANT (ANTIGRAVITY)**

Once I push it, the repository will be immediately accessible in your browser at:

👉 [<https://github.com/Jeevanthikashyap/buildwithgemini-yatra-ai>]  
(<https://github.com/Jeevanthikashyap/buildwithgemini-yatra-ai>)

You will be able to click that link directly to browse all your project code, the **README.md**, the looping demo GIF, and frontend files on your own GitHub account.

Before I push, please confirm: **Are you good for me to create the public repository and push the project now?**

**TURN 73 — USER**

yes please do it, push all the files each and everything included the video generated and everything we have created so far

**TURN 74 — ASSISTANT (ANTIGRAVITY)**

🎉 Your project is live on your personal GitHub!

**1. View Your Repository**

You can view and browse all your code, generated media, demo video, and looping demo GIF right here:

👉 [https://github.com/Jeevanthikashyap/buildwithgemini-yatra-ai]

(https://github.com/Jeevanthikashyap/buildwithgemini-yatra-ai)

Everything created during the session has been committed and pushed:

- `README.md` with embedded looping [ `demo.gif` ](https://github.com/Jeevanthikashyap/buildwithgemini-yatra-ai/blob/main/demo.gif)
- `demo_video.mp4` (Full 57-second demo recording)
- `app/` : ADK agent with Vertex AI Memory Bank, Firestore, Google Maps, Omni video generation, Imagen postcards, and A2UI v0.8
- `frontend/` : Complete FastAPI A2A proxy and styled web chat interface
- **Deployment files**: Terraform configs, seed scripts, and manifest

---

**2. Submit for Swag & Gallery**

Here is your pre-filled submission form with your repository URL already attached:

👉 [Submit Project for Swag & Gallery Entry]

(https://docs.google.com/forms/d/e/1FAIpQLSfvbIUMrHLf2iUYVgQkr981unQwuLdigLB7yJp3VdtYH85Dzw/viewform?usp=pp\_url&entry.896374137=https%3A%2F%2Fgithub.com%2FJeevanthikashyap%2Fbuildwithgemini-yatra-ai)

Open that link and fill in the final personal details:

1. Your **registration name & email**
2. Whether you've **claimed your GDP badge**
3. Whether you **consent to be featured in the showcase gallery**

**TURN 75 — USER**

so i wanted to ask you a few things about this project:

how long will the api key, cloud storage, firestore elements, cloud deployment elements, or any models we have used that are currently accessible and free stay free and accessible?

if i run out of these in future when i want present this project to some company , then how should i make it work?

**TURN 76 — ASSISTANT (ANTIGRAVITY)**

Here is an honest breakdown of **what happens to the cloud resources, how long they stay active, and how you can make this project run anytime in the future** when presenting it.

---

**1. How Long Will the Current Cloud Elements Stay Active?**

Because this workshop was run inside a **temporary Google Cloud lab environment (Qwiklabs / Google Cloud Skill Boost)**:

- **The Current Lab Workstation & Endpoints:**
- **Duration:** Only as long as your lab session remains active (typically a few hours or until the lab timer expires).
- Once the lab ends, the temporary GCP project ( `qwiklabs-gcp-04-8017249fc17a` ), the Cloud Run service, the deployed Agent Engine runtime, Firestore, and the Cloud Storage bucket are automatically deleted by Google.
- **The Code and Demo Assets:**
- **Permanent!** Because we just pushed everything to your personal GitHub repo ( [ `Jeevanthikashyap/buildwithgemini-yatra-ai` ](https://github.com/Jeevanthikashyap/buildwithgemini-yatra-ai)), all your code, the full `demo_video.mp4` , and the looping `demo.gif` are yours forever.

---

**2. How to Make It Work Later (Presenting to a Company or Client)**

You have **two great approaches** depending on how you want to present:

#### Option A: Presenting with Zero Cloud Setup (The "Portfolio/Demo Mode")

If you just want to showcase the project in an interview or meeting without spinning up cloud infrastructure:

1. **Show your GitHub Repository:** Open [ `github.com/Jeevanthikashyap/buildwithgemini-yatra-ai` ] (https://github.com/Jeevanthikashyap/buildwithgemini-yatra-ai).
2. **Play the Looping GIF & Video:** The `README.md` immediately showcases the agent planning a 3-day trip, executing Firestore database lookups, and generating the sunset postcard.
3. **Walk through the Architecture:** Explain the ADK patterns, A2A protocol, Vertex AI Memory Bank, and multi-modal tool calling in [ `app/agent.py` ](https://github.com/Jeevanthikashyap/buildwithgemini-yatra-ai/blob/main/app/agent.py). Most hiring managers and technical leads care most about

<truncated 847 bytes>

e & Cloud Storage (Free Tier)

1. **Firestore:** Create a Firestore Database in **Native Mode**. Run `python seed_firestore.py` with your new project ID to populate the curated spots. (Firestore's free tier gives you 1 GiB of storage and 50,000 reads/day for free forever).
2. **Cloud Storage:** Create a bucket with public read access (e.g., `gs://my-yatra-ai-media` ), or set `GCS_BUCKET_NAME` to your bucket.

##### Step 3: Update 2 Configuration Strings in `app/agent.py`

In `app/agent.py` , replace the two lab-specific strings with your own:

```
python

FIRESTORE_PROJECT_ID = "your-gcp-project-id"

GCS_BUCKET_NAME = "your-gcs-bucket-name"
```

##### Step 4: Run Locally with the ADK Playground

You don't even need to deploy to Cloud Run to demo it live! You can run it on your laptop:

```
bash
```

```
git clone https://github.com/Jeevanthikashyap/buildwithgemini-yatra-ai.git
```

```
cd buildwithgemini-yatra-ai
```

```
uv sync
```

## Authenticate your terminal to your personal GCP project

```
gcloud auth application-default login
```

```
gcloud config set project your-gcp-project-id
```

## Launch the ADK Dev UI

```
uv run adk web . --port 8080
```

```
`
```

Open `http://localhost:8080` in your browser and you have the live agent with all tools, image generation, and Firestore queries ready to interact with live.

```
##### Step 5: (Optional) Redeploy with agents-cli
```

If you want the production Cloud Run frontend and cloud deployment:

```
bash
```

```
uv run agents-cli deploy --project your-gcp-project-id
```

```
`
```

```
---
```

## Cost Summary for Personal Use

- **Gemini 2.5/3.x Flash & Flash-Lite:** Extremely inexpensive (fractions of a cent per prompt, often covered by Google AI Studio free tier limits or GCP's monthly free quotas).
- **Firestore:** Free tier includes 50,000 document reads per day.
- **Cloud Storage:** Free tier includes 5 GB per month.
- **Google Maps API:** Google grants a **\$200 free credit every month** automatically for Maps, Places, and Geocoding APIs.

In short, running this project for demonstrations on your own account will cost **effectively \$0** or pennies per demo session.

## TURN 77 — USER

now i want you to give me a pdf of the entire chat history, along with all the conversations we had and all the discussions we had so far, along with every steps, points covered and basically everything that is generated in this chat into a document for later demo purposes and uses, please cover this entire chat history